WEB422 Assignment 1

Assessment Weight:

10% of your final course Grade

Objective:

To work with a public "Books" API (https://openlibrary.org/dev/docs/api/books) on the client-side to produce a rich user interface for accessing data. For this assignment, we will be leveraging our knowledge of React and Next.js to create an interface for *viewing* books. **Please Note**: Once You're free to style your app however you wish. The following specification outlines how we can use the React-Bootstrap Components (Bootstrap 5). If you wish to add additional images, styles or functionality, please go ahead.

Step 1: Creating a Next App & Adding 3rd Party Components

The first step is to create a new directory for your solution, open it in Visual Studio Code and open the integrated terminal:

- Next, proceed to create a new Next.js app by using "create-next-app" with the command "npx create-next-app@latest" followed by the identifier for your app, ie: "my-app" and the "--use-npm" option.
- Once the tool has finished creating your Next App, be sure to "cd" into your new app directory and install the following modules using npm:
    - swr
    - bootstrap react-bootstrap
- To ensure that our newly-added Bootstrap components render properly, we must import the correct CSS file at the top of our "pages/_app.js" file, ie:
    - import 'bootstrap/dist/css/bootstrap.min.css';
- Next, we must delete (or clear the CSS from) the "Home.module.css" file and clear the existing CSS from "globals.css"
    - **NOTE:** if you want to have your table rows cause the cursor to change to a "pointer" on hover, the following CSS can be used here: **.table-hover tbody tr:hover{ cursor: pointer; }**

Step 2: Component "Skeletons"

For the next part of the assignment, we'll add our "page" / custom components (with some default text) as well as create a layout for our app.

NOTE: Components that output a simple … element will be overwritten further in the assignment

To proceed, add the following components:

- **Home (pages/index.js)** – Remove existing JSX and all imports and modify this to output Books
- **About (pages/about.js)** – outputs About
- **Work (pages/works/[workId].js)** – outputs Work (Book) by workId
- **MainNav (components/MainNav.js)** – outputs MainNav
- **Layout (components/Layout.js)** – outputs:

```
<MainNav />
<br />
<Container>
  {props.children}
</Container>
<br />
```

  NOTE: the "Container" is from "react-bootstrap", ie: import { Container } from 'react-bootstrap';

- **BookDetails (components/BookDeails.js)** – outputs BookDetails
- **PageHeader (components/PageHeader.js)** – outputs PageHeader

Step 3: App Component & NavBar

Before we start building out our components, we must first do some work with the "App" component (pages/_app.js):

- To begin, add the following import statements
  - import Layout from '**@/components/Layout**';
  - import { SWRConfig } from '**swr**';
- Next, add the following "fetcher" function **above** the App function:

```
const fetcher = async (...args) => {
  const response = await fetch(...args);

  if (!response.ok) {
    throw new Error(`Request failed with status: ${response.status}`);
  }

  return response.json();
};
```

- Once this is complete, place the <Component {...pageProps} /> inside our <Layout></Layout> component
- Finally, place the <Layout></Layout> component inside the following SWRConfig component to make our fetcher available, globally: <SWRConfig value={{ fetcher }}></SWRConfig>
- When complete, your App component should be returning:

```
<SWRConfig value={{ fetcher }}>
```

```
  <Layout>
    <Component {...pageProps} />
  </Layout>
</SWRConfig>
```

We can now create our "MainNav" component, so that our navbar shows more than just MainNav :

To begin, we should (at a minimum) have the following components imported:

- **Container, Nav, Navbar** from "react-bootstrap"
- **Link** from "next/link"

Next, use the documentation from the "Navbars" section of the React-Bootstrap docs (https://react-bootstrap.github.io/docs/components/navbar) to obtain the JSX code for a functioning Navbar according to the following specification:

- Shows ***Student Name*** in the **Navbar.Brand** element (where ***Student Name*** is your name) without an href element (ie: remove href="#home" from **Navbar.Brand**).

- The **Navbar** element should have a "className" value of "fixed-top". You can also use "navbar-dark" and "bg-dark" to achieve a design that matches the example.

- Contains 2 <Nav.Link> elements with the text "**Books**" and "**About**", linking to "**/**" (for **Books**) and **"/about**" for (**About**)

    - **NOTE:** For these elements to function correctly, they must use the "as" attribute. For example, the "Books" Nav.Link element should be:

      <Nav.Link as={Link} href="/">Books</Nav.Link>

- There should be **two** line breaks (<br />) after the **Navbar** element – this will ensure that the content can be seen below the *fixed* **Navbar**

- Once complete, you should have a responsive Navbar that looks similar to the following image (with your own Name in place of **Student Name**.

Step 4: Page Header Component (PageHeader.js)

With the navbar and other components in place we can concentrate on building a reusable "Page Header" element for our various views according to the following specification:

- The component must accept a custom property (prop), **text** (ie: <PageHeader text="some text" />

- The component must render the text prop in some kind of container. To achieve the same design as the example, a "Card" (see: https://react-bootstrap.github.io/docs/components/cards) was used with the className "bg-light" and the text value rendered within a <Card.Body>…</Card.Body> element (**NOTE:** <Card.Title>, <Card.Text> and <Card.Image> was not used and the card width was not set)

- There should be a line break (<br />) after the container

Step 5: About Component (about.js)

This component is primarily responsible for displaying information about the developer, including some information about a specific book available from our "Books" API.

To begin, we should (at a minimum) have the following components imported:

- **Link** from "next/link";
- **Card** from "react-bootstrap";
- **BookDetails** from "@/components/BookDetails";
- **PageHeader** from "@/components/PageHeader";

Additionally, this component must make use of "getStaticProps" (See "Fetching API Data for Pre-Rendered HTML" in the notes) to fetch a specific book from the API.

> **HINT:** This will involve figuring out what the "Work ID" is for a book that you would like to show. To explore the dataset for a book that you would like to use, you can browse https://openlibrary.org/. Once you have found a book, scroll to the "Book Details" section to see a unique "Work ID"

1. Once you know the Work ID value of the book you would like, make a "fetch" request within your promise-based "getStaticProps" function to fetch the book from the API using the path: "https://openlibrary.org/works/**(Work ID)**.json". For example: https://openlibrary.org/works/OL453657W.json will give you the data for the first "Discworld" Novel (Work ID: OL453657W).

2. Once the operation is successful, resolve the promise with an object containing the "props" keyword, with the data for your book (returned from the "fetch" request"), ie: **{props: {book: data}}**

With our "getStaticProps" function complete, we can now ensure that our component renders the following:

- A <PageHeader /> element with the text "About the Developer" followed by your name.

- A short description of yourself and the book you chose in one or more paragraphs.

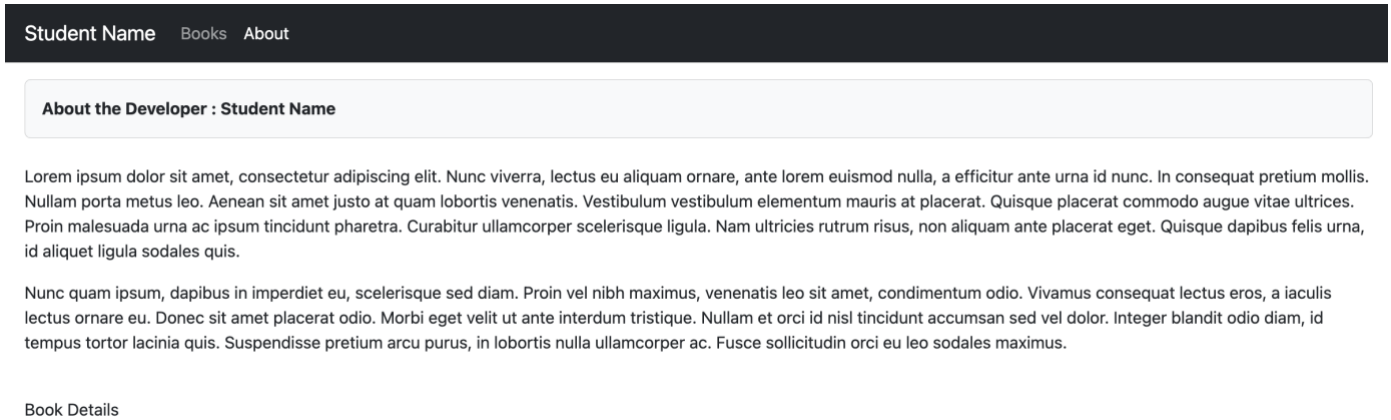
- A "BookDetails" element with a "book" property containing the data from your "getStaticProps" function, ie:

  <BookDetails book={props.book} />

  (**NOTE**: we will build this component further down in the assignment)

Step 6: Book Details Component (BookDetails.js)

At the moment, our "About" page should look something like this:



To complete this page, we should build the reusable "BookDetails" component next. As we have seen above, this component should accept the prop "book", which contains a full book object from the API. It is the job of this component, to render the data according to the following specification:

To begin, we should (at a minimum) have the following components imported:

- **Container, Row, Col** from "react-bootstrap"

Next, we must ensure that the component renders a single React Bootstrap "Container" element, containing one "Row" element, with one <Col lg="4"></Col> Element and one <Col lg="8"></Col> Element. Specifically:

- In the first "Col" (<Col lg="4">) element (ie: the left column), we will show an image of the book using following code (with optional line-breaks beneath it, for spacing), where **book** is the full object sent in the "book" prop:

```
<img
  onError={(event) => {
  event.target.onerror = null; // Remove the event handler to prevent infinite loop
  event.target.src =
    "https://placehold.co/400x600?text=Cover+Not+Available";
  }}
  className="img-fluid w-100"
  src={`https://covers.openlibrary.org/b/id/${book?.covers?.[0]}-L.jpg`}
  alt="Cover Image"
/>
<br />
<br />
```

- Next, html must be rendered within a second "Col" (Col lg="8") element to show book information. For example, if we use book: "The Colour of Magic" (ie: "https://openlibrary.org/works/OL453657W.json"), the following HTML must be generated (again, please free to change the style, if it fits your design).

  <h3>**The Colour of Magic**</h3>
  Terry Pratchett's profoundly irreverent novels are consistent...

  <h5>Characters</h5>
  Samuel Vimes (Fictitious character), Rincewind, Twoflower (fictional characters), Twoflower

  <h5>Settings</h5>
  Discworld (Imaginary place), Discworld, Ankh Morpork

  <h5>More Information</h5>
  **<a href="http://en.wikipedia.org/wiki/Discworld" target="_blank">Wikipedia (Discworld)</a>
**
  **<a href="https://www.goodreads.com/book/show/601238.The_Colour_of_Magic" target="_blank">Goodreads</a>
**

  The HTML shown above can be obtained using the following properties from the book data:

  - **The Colour of Magic** – obtained from the **title** property

  - **Terry Pratchett's profoundly irreverent novels are consistent...** – obtained from the **description.value** property
    - **NOTE:** Not all books have a **description** or **description.value** property. One way of solving this is to use the following JSX:

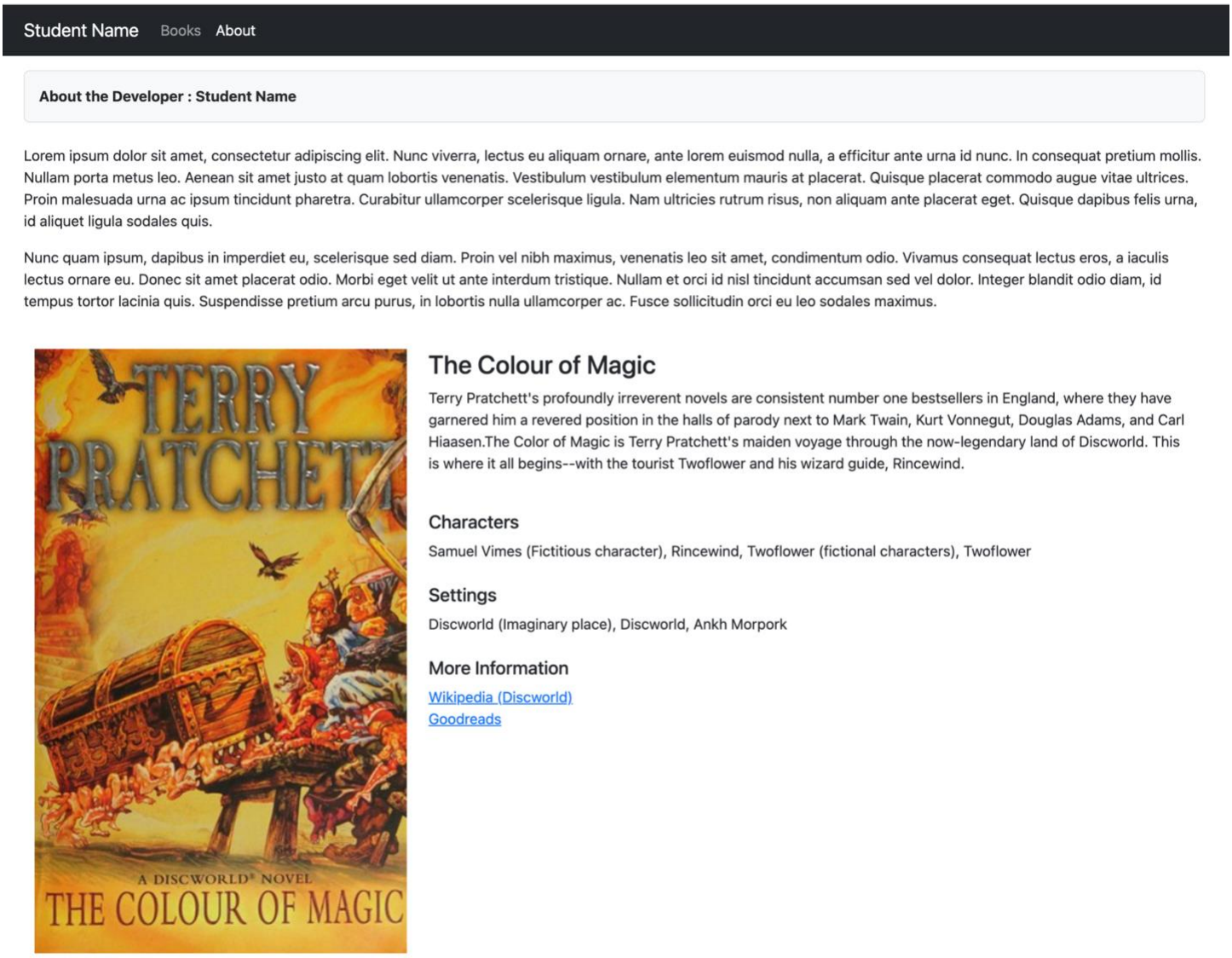
      ```
      {book.description && (
        <p>
          {typeof book.description === "string" ? book.description : book.description.value}
        </p>
      )}
      ```

  - **Samuel Vimes (Fictitious character), Rincewind, Twoflower (fictional characters), Twoflower** – obtained from **joining** the array stored in the **subject_people** property (**NOTE:** Not all books have **subject_people**)

  - **Discworld (Imaginary place), Discworld, Ankh Morpork** – obtained from joining the array stored in the **subject_places** property (**NOTE:** Not all books have a **subject_places**)

  - **<a href="http://en.wikipedia.org/wiki/Discworld" target="_blank">Wikipedia (Discworld)</a>
<a href="https://www.goodreads.com/book/show/601238.The_Colour_of_Magic" target="_blank">Goodreads</a>
** – obtained by looping through the array stored in the **links** property (**NOTE:** Not all books have **links**)

Once you have completed this component, refresh your "About" page. It should show the book that you obtained from your "getStaticProps" function, since it was provided to "BookDetails", via the "book" prop. For example, if the book data for "The Colour of Magic" was obtained in the "getStaticProps" function:



Step 7: Home Component (index.js)

We now have all the pieces in place to build our "Home" component. This is arguably one of the more complicated components as it must display data from the "Books" API, one page at a time.

To begin, we should (at a minimum) have the following components imported:

- **useSWR** from 'swr';
- **useState, useEffect** from 'react';
- **useRouter** from 'next/router';
- **Pagination, Table** from 'react-bootstrap';
- **PageHeader** from '@/components/PageHeader';

Next, add the following **state** values to the component:

- **page** (default value: 1)
- **pageData** (default value: [])

Also, add a **router** instance:

- const **router** = useRouter();

Finally, add an **author** constant with the text of the author that you wish to feature in your application, for example:

- const **author** = 'Terry Pratchett';

With the values in place, we can now use SWR to make a request for the data, ie:

- const { data, error } = useSWR(`https://openlibrary.org/search.json?author=${encodeURIComponent(author)}&page=${page}&limit=10`);

  NOTE: You do not need to provide a "fetcher" here, since it was defined globally using the SWRConfig component (above)

We will be using the data returned from useSWR to set our "pageData" state value, rather than using it directly within our JSX for this component (this will prevent flickering while the data loads). To ensure that whenever the "data" has been successfully updated (as a result of a new page being requested, for example), we can leverage the "useEffect" hook as follows (assuming "setPageData()" is your setter function for your **pageData** state value):

```
useEffect(() => {
  if (data) {
    setPageData(data);
  }
}, [data]);
```

Finally, before we can write the return statement (JSX) for this component, we must ensure that we have two functions declared:

- **previous** – decreases the **page** state value by **1** if it is greater than **1**
- **next** – increases the **page** state value by **1**

As you have seen from the example, the output for this component is a Table element, featuring book information (title & published date). Book data can be obtained one page at a time, where each page shows 10 items. To create this UI, we must include:

- A <PageHeader /> element with the text "Novels by **(Author)**", where **(Author)** is the value of your **author** constant

- An <Table striped hover> element containing one <thead> (for the headings) and one <tbody> element, containing <tr> elements for each **book** in the **pageData** array according to the following specification:

- When clicked the <tr> elements must invoke the **router.push()** function with the value of **book.key** (this will cause the app to navigate to the correct book)

- Both <td> elements within the <tr> element must show the values of:

  - The current book's "title" value
  - The current book's "first_publish_year" value (Note: If this value is missing, show "N/A" instead)

- A Pagination element (<Pagination>) containing the following elements:

  - <Pagination.Prev /> with a "click" event that executes the "previous" function
  - <Pagination.Item></Pagination.Item> which shows the current **page** value
  - <Pagination.Next /> with a "click" event that executes the "next" function

Step 8: Work Component (works/[workId].js)

The final component that we must create is the dynamic "Work" component. The purpose of this component is to show a **<BookDetails />** component for a specific book, specified by the "workId" route parameter. If a book cannot be found with a matching workId, the default Next.js error is shown.

To begin, we should (at a minimum) have the following components imported:

- **useRouter** from 'next/router';
- **useSWR** from 'swr';
- **BookDetails** from '@/components/BookDetails';
- **Error** from 'next/error';
- **PageHeader** from '@/components/PageHeader';

Before we can make our request (using SWR) we must obtain the "workId" parameter (**HINT:** this can be done with the useRouter(); hook – see "Reading Route Parameters" from the course notes)

Next, using the "workId" parameter value, use SWR to make a request for the data, ie:

- const { data, error, isLoading } = useSWR(`https://openlibrary.org/works/${workId}.json`);

Before we can render anything however, we must ensure that we are not currently "loading" (SWR) the data. If this is the case, we can just return **null**.

Otherwise, if we're not loading the data, we must make sure that we don't have an "error" (SWR) or that the "data" returned is valid (doesn't have a null value / empty array, etc). If this is the case, we should return the component: **<Error statusCode={404} />** (this will cause the default Next.js 404 error to be displayed).

However, if everything was successful (loading is complete, there were no errors / empty or null data, etc), then we should render the following:

- A <PageHeader /> element with book's "title" property.

- A <BookDetails /> element with a "book" property containing the full book object
  <BookDetails book={data} />

Assignment Submission:

- Add the following declaration at the top of your index.js file

```
/*****************************************************************************
*  WEB422 – Assignment 1
*
*  I declare that this assignment is my own work in accordance with Seneca's
*  Academic Integrity Policy:
*
*  https://www.senecapolytechnic.ca/about/policies/academic-integrity-policy.html
*
*  Name: _____ Student ID: _____ Date: _____
*
*****************************************************************************/
```

- Compress (.zip) the files in your Visual Studio working directory **without node_modules.** This is the folder that you opened in Visual Studio to create your Next.js App.

Important Note:

- **NO LATE SUBMISSIONS** for assignments. Late assignment submissions will not be accepted and will receive a **grade of zero (0)**.

- Submitted assignments must run locally, ie: start up errors causing the assignment/app to fail on startup will result in a **grade of zero (0)** for the assignment.